

SmartAgent Receiver

Splunk OpenTelemetry Collector

What it is, why it needs Docker access, and how it runs in a container

What Is the SmartAgent Receiver?

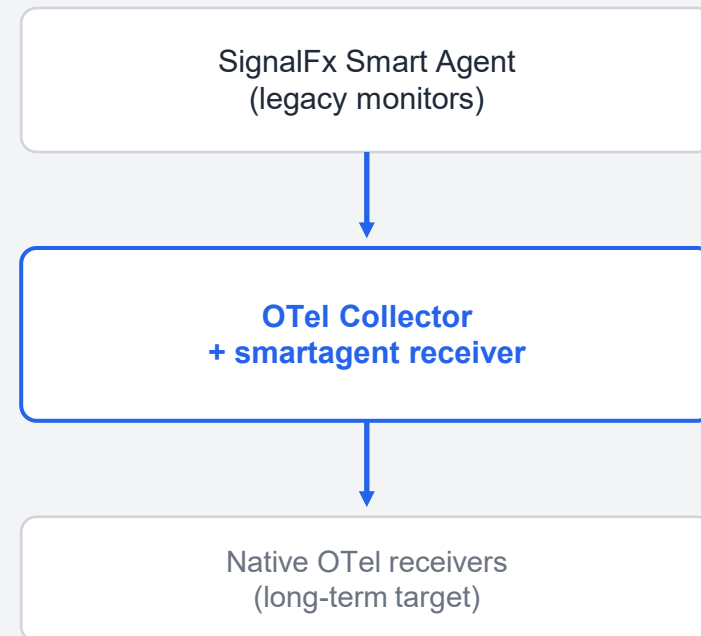
A compatibility bridge, not a new agent

It is a receiver built into the Splunk OTel Collector binary that lets legacy SignalFx Smart Agent monitors (MySQL, Redis, Kafka, Docker stats, process list, etc.) keep running inside the new OTel pipeline.

Why it exists

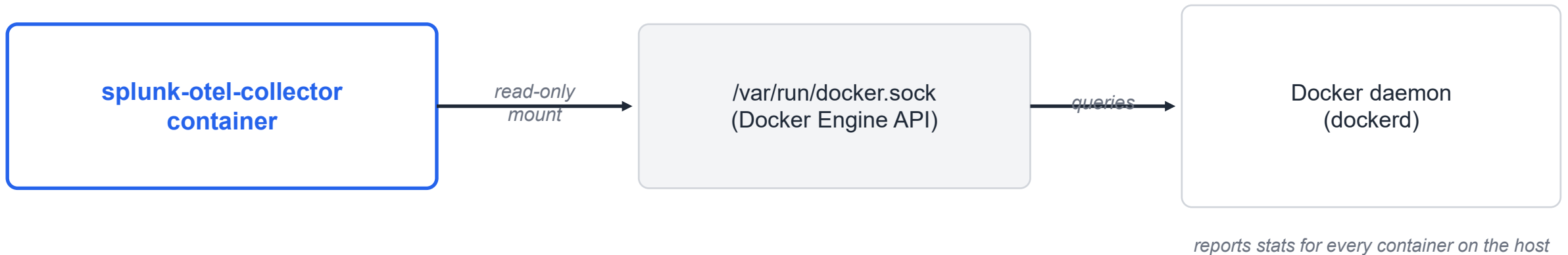
Many Smart Agent monitors were never ported to native OTel receivers. The SmartAgent receiver avoids rewriting every monitor config during migration — it is a drop-in replacement for its corresponding Smart Agent monitor.

Migration path



Why Does It Need Docker Access?

The smartagent/docker-container-stats monitor reads per-container CPU, memory, and network stats directly from the Docker Engine API — the only place those numbers exist.



1. Permission on the socket

The collector user (or process) must be a member of the docker group, or run with the socket's group ID, to read `/var/run/docker.sock`.

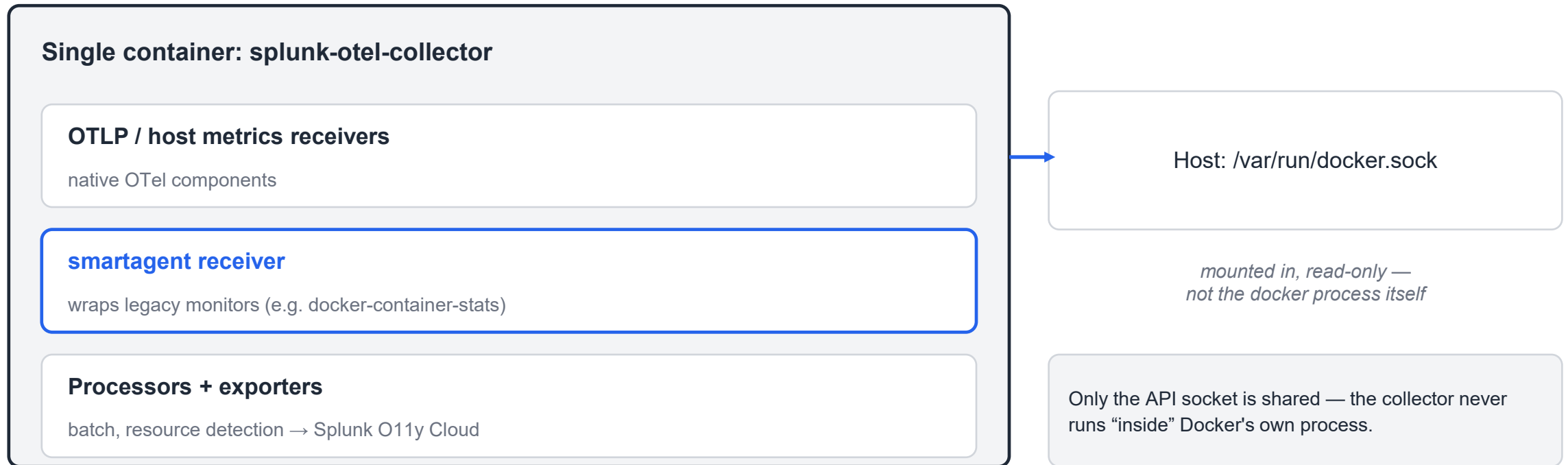
2. Volume mount into the container

When the collector itself runs as a container, the host socket is bind-mounted in (read-only) so it can reach the daemon.

Does It Fit Inside One Container?

Yes — it is one process, not a separate agent

The smartagent receiver is compiled into the same splunk-otel-collector binary/image. Collector + SmartAgent receiver + Docker monitor all run as a single process, inside a single container (or host service).



Key Takeaways

What it is	A built-in Collector receiver that runs legacy Smart Agent monitors, avoiding a config rewrite during migration to OTel.
Why Docker access	The docker-container-stats monitor must query the Docker Engine API for per-container metrics — that data only lives there.
What access, exactly	Read access to the Docker socket (/var/run/docker.sock) or TCP endpoint — group membership or a mounted volume, not root on the host.
Container fit	Yes: receiver, monitor, and Collector are one process in one container; only the socket is shared in from the host.